uniRAG: An Offline Retrieval-Augmented Study Assistant with Deterministic Exam Analytics for University Students

Raghavendra Chandrakantha Bhat
Department of Computer Science and Applications, Bangalore University, Bangalore, India
raghavendra.c.bhat@gmail.com

***Abstract*—Examination preparation at universities depends on scattered notes and scanned question papers, and students cannot easily learn how often a topic was asked, in which years or for how many marks. Cloud chatbots do not solve this, because they need the internet, send questions to external servers, ignore the university syllabus and can invent facts such as frequency counts. This paper presents uniRAG, an offline study assistant for the Master of Computer Applications programme of Bangalore University that runs on a single CPU-only laptop. It combines retrieval-augmented generation, a locally hosted Mistral 7B model, per-subject FAISS indexes and Tesseract OCR with a question bank of 1,426 questions extracted from scanned papers. The key idea is that the language model only writes explanations, while deterministic code computes every count, year, mark value and trend, and guards decline questions the notes do not support. Calibrating the semantic matching cut-off on 61 labelled queries raised the F1 score from 0.435 to 0.757. Exam-fact answers take about 25 s and explanations one to three and a half minutes without a GPU. The results show that a private, grounded and low-cost study assistant is practical on ordinary student hardware.**

***Index Terms*—retrieval-augmented generation, offline language models, educational technology, question-paper analysis, vector search, OCR, hallucination control**

I. INTRODUCTION

India has one of the largest higher-education systems in the world: the All India Survey on Higher Education 2021-22 recorded about 4.33 crore (43.3 million) enrolled students, up from 4.14 crore a year earlier [1]. For most of them, examination preparation means reading notes and past question papers, often scanned images, and the questions a student really needs answered are practical ones: what does this topic mean, how many times has it been asked, in which years, and for how many marks. Answering them by hand means reading dozens of pages, and the result is slow and rarely complete.

Large language models (LLMs) can explain almost any topic, but using a cloud chatbot for this purpose has three problems. It needs an internet connection and sends every question to an external server. It has no knowledge of what a particular university teaches or asks. And it can hallucinate, that is, state wrong facts, or invent numbers, with full confidence [4], [5]. Retrieval-augmented generation (RAG) reduces the second and third problems by giving the model passages retrieved from a trusted document store [2], [3], but a RAG system that lets the model report exam statistics still risks invented counts, because counting is not something a language model does reliably.

This paper describes uniRAG, built for the first three semesters of the Master of Computer Applications (MCA) programme of Bangalore University. The contributions are:

- A fully offline RAG architecture (Python, Streamlit, Ollama, FAISS, Tesseract) that runs on a 16 GB CPU-only laptop and keeps all student data on the machine.
- A *model writes, code counts* design: all frequencies, years, marks and trends come from a structured question bank of 1,426 questions extracted from scanned papers, never from the language model.
- A four-level hybrid question matcher and a set of guards against unsupported answers, with the semantic cut-off chosen by measurement on a labelled set rather than by guess.
- An evaluation on the real university material: question-bank statistics, matching quality at different cut-offs, response times by question type, a regression suite and worked examples.

The rest of the paper is organised as follows. Section II reviews recent related work. Section III presents the architecture and Section IV the implementation. Section V explains the methods in detail, Section VI describes the experimental setup, and Section VII reports and discusses the results. Section VIII lists limitations and Section IX concludes.

II. LITERATURE REVIEW OF RECENT RESEARCH

This section reviews fifteen recent research works, from 2024 to 2026 unless stated otherwise, that are related to uniRAG. They are grouped by the problem they address: retrieval-augmented generation and its reliability, RAG in education, offline and local RAG, questions that need counting, OCR quality and Indian-language retrieval.

A. Foundations: RAG and hallucination

RAG combines a retriever with a generator so that the output is conditioned on retrieved text [2]. A recent

survey divides RAG systems into naive, advanced and modular forms and describes retrieval, generation and augmentation techniques together with evaluation methods [3]. Another survey classifies LLM hallucinations into factuality and faithfulness errors, traces their causes to data, training and inference, and notes that even retrieval-augmented models still hallucinate and that the limits of a model's knowledge are hard to determine [4]; the earlier survey of Ji *et al.* reached similar conclusions for text generation [5]. These works motivate uniRAG's choice to give the model only the task of writing from retrieved text, and to guard against unsupported output instead of trusting it.

B. Improving and evaluating RAG

Self-RAG trains a language model to decide when to retrieve and to critique its own generations with special reflection tokens, which improves factuality and citation accuracy over standard RAG [6]. uniRAG pursues the same goal without training, by using deterministic guards before and after retrieval. For evaluation without human-written reference answers, the Ragas framework offers metrics for the retrieval component, for how well the model uses the retrieved context, and for the quality of the output [7]. Li *et al.* compared long-context models with RAG on 12 question-answering datasets expanded to about 19,000 questions: long-context models generally won, especially on Wikipedia and story material, but RAG did better on dialogue sources and general yes/no questions, so the two approaches are complementary [8]. For a small model on a CPU with a corpus of thousands of chunks, long contexts are impractical, which supports retrieval over a per-subject index.

C. RAG for education

Lang and Gurpinar built a chatbot that answered student questions in a self-paced online R-programming course from the course materials; interaction logs and surveys showed that students with stronger prior knowledge used it most, often for topics beyond the lectures, and that most students were highly satisfied, especially with answers grounded in the course content [9]. Kahl *et al.* evaluated prompt engineering, RAG and fine-tuning for a robotics-course tutor on 478 test chats: for GPT-3.5, RAG with a system message raised BLEU-4 from 0.03 to 0.10 and ROUGE-1 from 0.22 to 0.43, while a fine-tuned 13B LLaMA-2 without RAG matched it on similarity metrics and combining fine-tuning with RAG degraded results; they also found that common similarity metrics favour short answers and need improvement [10]. Ostrowska *et al.* describe a Moodle plugin that grounds a Socratic tutor in teacher-supplied material and lets teachers approve generated quizzes; evaluated with Ragas it reached a faithfulness of 0.97 for STEM subjects, and 18 participants rated its resistance to hallucination at 4.44 out of 5, though the study was preliminary [11]. These systems answer questions about course content. None of them computes examination statistics, such as how often a topic was asked in past papers, which is the second main function of uniRAG.

D. Offline and local RAG

Takamura and Umezawa built PRISM, an offline RAG system for regenerative-medicine questions that runs a 12B-parameter Gemma 3 model and a FAISS index on a MacBook Air without a GPU; on 30 question-answer trials, binary accuracy rose from 60 % without retrieval to 86.7 % with retrieval, with response times of 3.1 to 5.7 s, although the test was small and no user study was done [12]. Lee describes an offline RAG prototype for natural-disaster safety advice that runs sub-billion-parameter models on a CPU, found that few-shot examples in the prompt contaminated the context, and cut latency from more than 60 s to about 5–9 s by persisting the vector index [13]. Devine proposes ALoFTRAG, which lets a local LLM generate its own synthetic question-answer data and fine-tune with LoRA, raising citation accuracy by 8.3 % and answer accuracy by 3.0 % on average over 20 datasets in 26 languages while keeping all data local [14]. Dayarathne *et al.* compared GPT-3.5 with four open-source 7B models, including Mistral-7B-Instruct, in a RAG pipeline over 4,929 computer-science abstracts: GPT-3.5 with RAG was best (0.905 accuracy on binary questions), Mistral-7B-Instruct was the best open model (0.857), and the open models were free but much slower, at roughly 99 to 107 s per answer against 1.7 s for GPT-3.5 [15]. These results show that a 7B open model is accurate enough for grounded question answering, and that its price is latency, which matches what we measured. uniRAG follows the same local-first principle, persists per-subject indexes and an answer cache for the same latency reason, and uses a prompt without few-shot examples.

E. Counting and corpus-level questions

A question such as “how many times has this topic been asked?” is an aggregation over a whole corpus, and retrieval of a few top chunks cannot see every matching item. Luo *et al.* introduced GlobalQA, a benchmark of over 13,000 question-answer pairs on more than 2,000 resumes in 23 domains covering counting, extremum, sorting and top-k tasks, and showed that standard RAG reaches only 1.51 F1 with Qwen2.5-14B, while methods that score 17 to 41 F1 on multi-hop datasets fall below 2 F1 [16]. Zhu *et al.* introduced AGGBench, with 362 aggregation queries over 45 research papers, and reported that Text-to-SQL and top-k RAG often miss evidence and undercount, with evidence density falling to 1.09 % in the larger corpus; an agentic baseline raised evidence recall up to five times, although its counting errors were not the lowest [17]. These findings support uniRAG's central rule: counts and statistics must not be left to a language

reading retrieved chunks, but computed by code over a structured question bank.

F. OCR quality and RAG

Scanned papers are the main source of uniRAG's question bank, so OCR errors matter. Zhang *et al.* introduced OHRBench, with 8,561 document images and 8,498 question-answer pairs across seven RAG domains, and showed that the best OCR solutions still trail ground-truth text by at least 14 %, that semantic noise (wrong characters and reading order) hurts most, with drops of nearly 50 % at high noise levels, and that sparse and dense retrievers degrade similarly [18]. Sun *et al.* found with InduOCRBench that high OCR accuracy does not guarantee high RAG accuracy: one model reached 82.9 % OCR accuracy but only about 53 % RAG accuracy, and tables and formulas suffered most [19]. These results explain why uniRAG cleans scanner watermarks and page-turn marks, keeps OCR only as a fallback for pages with no text layer, and lists OCR noise as a limitation.

G. Indian-language retrieval

Students in India often mix English with Hindi or Kannada words. Prasanjith *et al*. released IndicRAGSuite, containing IndicMSMARCO (1,000 queries translated into 13 Indian languages and manually verified) and about 14 million question-answer-reasoning triplets in 19 languages, and found that the BGE-M3 retriever gave the best or near-best score in 8 of the 13 languages [20]. uniRAG currently handles only mixed-language wording that the model can parse (for example “queue kitni baar aaya hai exam me”); full Indian-language support is left to future work.

H. Research gap

Table I summarises the reviewed works. Education-oriented systems focus on answering content questions, offline systems demonstrate feasibility on consumer hardware, benchmarks show that counting and noisy OCR are weak points of RAG, and no reviewed work combines these concerns. What is not found among them is a local system for a closed university corpus that (i) turns scanned past papers into a structured question bank, (ii) produces counts, years, marks and trends by deterministic code so the model cannot invent them, and (iii) chooses its matching threshold by measurement. uniRAG addresses this gap.

TABLE I. Summary of reviewed work and uniRAG.

Work	Focus	Local / offline	Key result
[6] Asai, 2024	Self-reflective RAG	No	Better factuality, citations
[8] Li, 2024	Long context vs RAG	No	Complementary trade-offs
[9] Lang, 2025	RAG chatbot, R course	Not stated	High satisfaction
[10] Kahl, 2024	Tutor: RAG, fine-tuning	Partly	ROUGE-1 0.22→0.43
[11] Ostrowska, 2026	Moodle tutor	Not stated	Faithfulness 0.97
[12] Takamura, 2025	Medical RAG (PRISM)	Yes, laptop	Accuracy 60→86.7 %
[13] Lee, 2026	Disaster advice	Yes, CPU	Latency 60 s→5–9 s
[14] Devine, 2025	Local fine-tuning	Yes	Answer acc. +3.0 %
[15] Dayarathne, 2025	Open 7B vs GPT-3.5	Open models	0.857 vs 0.905 acc.
[16] Luo, 2025	Counting in RAG	No	Standard RAG 1.51 F1
[17] Zhu, 2026	Aggregation queries	No	Top-k RAG undercounts
[18] Zhang, 2025	OCR impact on RAG	–	Up to ~50 % drop
[19] Sun, 2026	OCR robustness	–	OCR acc. ≠ RAG acc.
[20] Prasanjith, 2025	Indian-language RAG	–	BGE-M3 best in 8/13
uniRAG (this work)	Study help + exam statistics	Yes, CPU laptop	F1 0.435→0.757

III. SYSTEM ARCHITECTURE

uniRAG has four layers (Fig. 1). The presentation layer is a Streamlit application with role-based login (student and administrator). The application layer is a set of Python modules for query understanding, retrieval, question matching, exam statistics and document loading. The local AI services are two Ollama models, Mistral 7B [23] for generation and parsing and nomic-embed-text [22] for 768-dimensional embeddings. The data layer holds the PDF library, per-subject FAISS [21] indexes, a JSON question bank and the student file. Nothing communicates with the internet at run time.

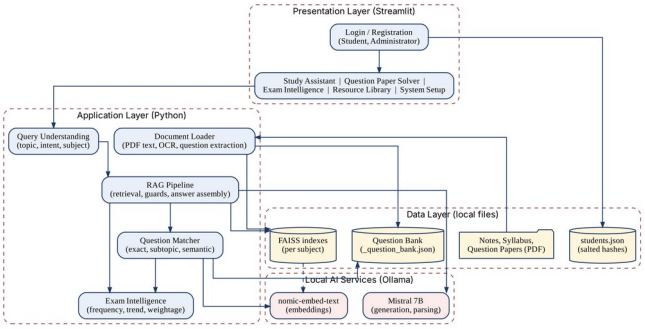


Fig. 1. System architecture of uniRAG.

A question follows the pipeline in Fig. 2: it is parsed into a structured query (topic, intents, subject) at temperature 0, evidence is retrieved from the notes and the question bank, guards decide whether the evidence is good enough, and only then is the model asked to write the explanation. The final answer is assembled from the model text and facts computed by code and is shown in separate cards (concept, exam history, trend, guidance, sources).

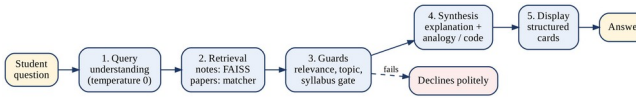


Fig. 2. Answer pipeline.

IV. IMPLEMENTATION

A. Technology stack

TABLE II. Technology stack of uniRAG.

Component	Tool	Role
Interface	Streamlit	Login, tabs, chat, uploads, charts
Models	Ollama; Mistral 7B	Parsing, explanations, lab code
Embeddings	nomic-embed-text	768-d vectors for notes and questions
Search	FAISS (faiss-cpu)	Per-subject vector indexes
Text splitting	LangChain	400-char chunks, 50 overlap
PDF / OCR	PyPDF, PyMuPDF, Tesseract	Text layer, rendering, scanned pages
Charts	pandas, Altair	Trend and ranking charts
Security	hashlib, hmac, secrets	Salted PBKDF2 hashes, lockout

B. Modules and data stores

The code is organised into modules for the user interface (`app.py`), configuration, document loading, query understanding, the RAG pipeline, the question index, the question matcher, exam intelligence, curriculum questions and repeated-question solutions. Structured data is kept in JSON files (student accounts, the question bank, a build manifest, the answer cache, the solutions store and university settings), while embeddings are kept in FAISS indexes, one per subject plus one for the questions. For this workload of about 1,400 questions written in bulk by a rebuild and then only read, JSON and FAISS need no database server, can be copied with the project folder and can be rebuilt from the source PDFs.

C. Application features

A student uses four tabs. The *Study Assistant* answers a free-text question with an explanation, an analogy, exam history, trend and guidance. The *Question Paper Solver* extracts every question of an uploaded paper, answers each through the same pipeline in a plain-answer mode and offers the answers as a download. *Exam Intelligence* gives the year-by-year trend of a topic, the most repeated questions with a solutions PDF and the high-weightage questions for a chosen marks level. The *Resource Library* lists the indexed documents. The administrator additionally sees a *System Setup* tab for uploading documents, setting the university name and rebuilding indexes. Lab programs are written in C by default and in Java for the object-oriented lab or when Java is asked for.

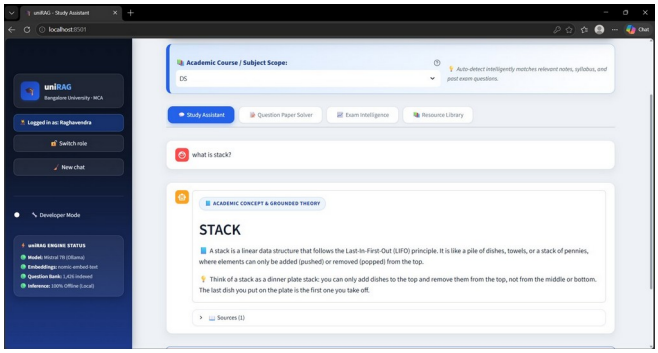


Fig. 3. Study Assistant answer with concept, analogy and sources.

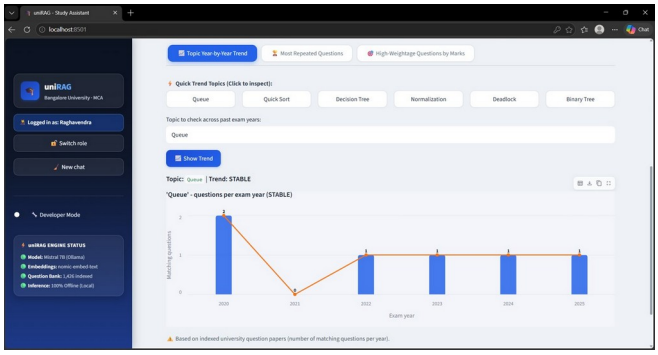


Fig. 4. Exam Intelligence: year-by-year trend of a topic.

V. METHODOLOGY

A. Document processing and question bank

Notes are split into chunks of 400 characters with a 50-character overlap and embedded into one FAISS index per subject; a single merged index mixed subjects and let large subjects crowd out small ones, so it is kept only as a fallback. The approximate number of chunks for a document of L characters, chunk size S and overlap V is

$$N \approx \lceil (L - V) / (S - V) \rceil \quad (1)$$

Pages with no text layer are read with Tesseract [24]. Question papers, in which several years are often merged in one scanned PDF, are read page by page; the exam year is taken from the printed header and carried over continuation pages, and the text is split into individual questions with subject, semester, year and marks. Scanner watermarks and page-turn marks are removed. The result is a question bank of 1,426 questions from 18 subjects and seven exam years (2020 to 2026), with a separate small vector index of the questions.

B. Query understanding

The model parses a messy question, which may contain spelling mistakes, abbreviations or mixed-language words, into a structured query holding the canonical topic, subtopics, intents (definition, frequency, exact questions, trend, ranking, lab program and so on), subject, marks and year constraints. Parsing runs at temperature 0, so the same question always parses the same way. Deterministic code then corrects what a small model gets wrong, for example a topic is carried over from the previous message only if the new question actually points back at it.

C. Hybrid question matching

Questions are matched at four levels: exact, subtopic, semantic (a dedicated question vector store) and filters on marks and year. A fuzzy match uses the difflib ratio, where M is the number of matching characters and T the total number of characters in both strings:

$$\text{ratio}(A, B) = 2M / T \quad (2)$$

A ratio of at least 0.75 counts as a near-exact match, above 0.8 as a subtopic match, and a topic word is spell-corrected to a word of the papers at a ratio of at least 0.82. The semantic level accepts a stored question only if its embedding distance from the query is below a cut-off. The FAISS flat index returns the squared Euclidean distance between the query vector q and a stored vector c:

$$d(q, c) = \Sigma\, (qi - ci)^2,\ \ i = 1 \ldots 768 \quad (3)$$

D. Guards

Before the model writes anything, the guards in Table III check that the evidence supports an answer. Declined questions never reach the model, so they are answered instantly.

TABLE III. Guards against wrong answers.

Guard	What it stops
Relevance cut-off (distance ≤ 1.15)	Answers built from unrelated text
Topic-in-context check	A topic answered from model memory
Out-of-syllabus gate	Definitions of topics absent from syllabus and papers
Deterministic counting	Invented frequencies or years
Follow-up check	Topic carried into an unrelated question
Figure-reference stripping	“Figure 3.1 shows…” the student cannot see
Empty-input guard	A bare “?” giving a random answer

E. Deterministic exam analytics

Counts, years and marks are read from the question bank. The year-by-year trend of a topic is classified by splitting the yearly counts into an earlier and a later half and comparing their averages with a margin of 0.5 question:

$$\text{rising if avg(later)} > \text{avg(earlier)} + 0.5 \quad (4)$$

with *falling* for the symmetric case and *stable* otherwise. The coverage of a topic is

$$\text{Coverage (\%)} = 100 \times D / Y \quad (5)$$

where D is the number of distinct exam years in which the topic appeared and Y the number of sittings on record for the subject. Questions asked in two or more sittings are listed as the most repeated questions, and a PDF bundles a model answer for each; 67 of the 125 answers were curated and 58 were generated offline by the local model.

F. Answer cache and security

A saved answer is looked up by a SHA-256 digest of the model, prompt, retrieved notes, question, marks guidance and token limit, so an identical input returns the stored answer in seconds; because a selected subject changes the retrieved notes, it also changes the key. Passwords are stored as PBKDF2-HMAC-SHA256 hashes [25] with a random salt and 100,000 iterations, compared in constant time, and five wrong attempts lock an account for 60 s. Data files are written to a temporary file and then atomically replaced, and a corrupt student file is never overwritten without a backup.

VI. EXPERIMENTAL SETUP

All measurements were made on a Lenovo IdeaPad Slim 3 (Intel Core i5-13420H, 16 GB RAM, no GPU, Windows 11) with Python 3.12, Ollama serving mistral:7b (answer temperature 0.1, parsing temperature 0) and nomic-embed-text, chunk size 400, overlap 50 and top-3 retrieval. The corpus consists of the notes, syllabus and question papers of 18 subjects of the first three semesters, indexed in 22 subject collections. The matching cut-off was calibrated on a labelled set of 61 queries: 46 with a correct stored question in a given subject and 15 for which nothing should match. For each cut-off from 0.30 to 1.20 the returned questions were compared with the answer key, and precision, recall and F1 were computed, where TP, FP and FN are the numbers of correct questions returned, wrong questions returned and correct questions missed:

$$P = TP/(TP+FP),\ \ R = TP/(TP+FN) \quad (6)$$

$$F1 = 2PR/(P+R) \quad (7)$$

A regression suite of 29 fixed cases with 44 checks, run through the same code path as the chat interface, verifies rules such as no topic carry-over, refusal of out-of-syllabus topics, consistent counts and years, code in the right language and identical answers for identical questions; it also records the response time of each question. In addition, 27 manual test cases cover registration, login, lockout, roles, the Question Paper Solver, Exam Intelligence, OCR, rebuild and a corrupt data file.

VII. RESULTS AND DISCUSSION

A. Question bank

The question bank holds 1,426 questions: 692 carry 6 marks (48.5 %), 336 carry 5 marks, 251 carry 10 marks, and the rest 3, 4 and other values (Fig. 5). The years 2022 to 2025 hold 274 to 326 questions each, while 2020 and 2026 hold 70 and 86 (Fig. 6), so year-wise statistics for those two years rest on fewer papers. Subjects differ widely in size, from 118 questions (QTR) to 14 (CNS), so counts for small subjects rest on a small sample. Across 16 subjects, 125 questions were repeated in two or more sittings (ML and CNS have none).

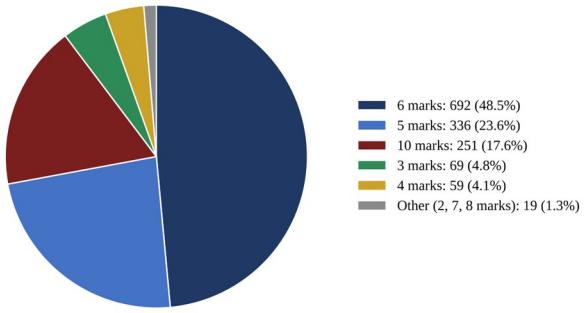


Fig. 5. Share of questions by marks allotted.

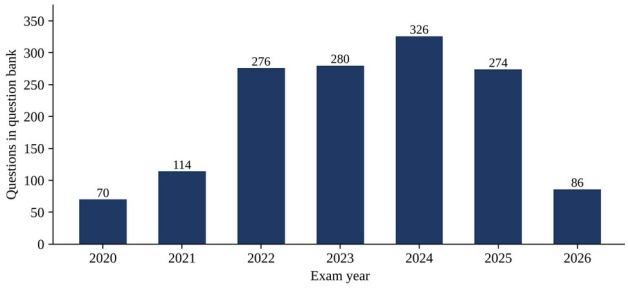


Fig. 6. Questions in the question bank per exam year.

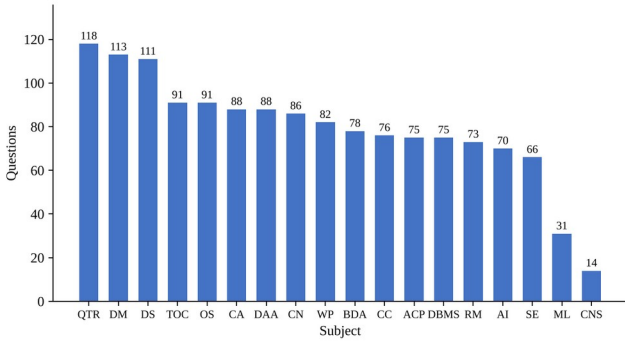


Fig. 7. Questions per subject.

The number of questions per exam year also differs between semesters: every semester has papers from five or six of the seven years, Semester 1 has no 2026 papers, Semester 2 has no 2021 or 2026 papers, and Semester 3 has no 2020 papers but is the only semester with 2026 papers. A year without papers shows as zero in the trend chart, so a zero may mean that no paper exists, not that the topic was not asked.

B. Matching quality

TABLE IV. Matching quality at selected cut-offs (61 queries).

Cut-off	Precision	Recall	F1	Nothing-queries wrongly answered
0.50	0.989	0.298	0.457	0/15
0.60	0.907	0.474	0.623	0/15
0.70	0.855	0.651	0.739	1/15
0.75	0.777	0.737	0.757	1/15
0.90	0.490	0.879	0.629	7/15
1.15	0.285	0.917	0.435	15/15

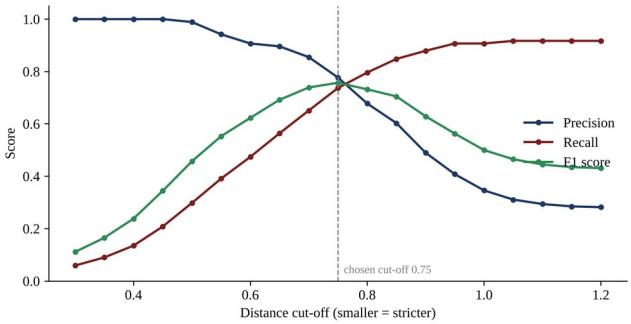


Fig. 8. Precision, recall and F1 against the distance cut-off.

Table IV and Fig. 8 show the expected trade-off. A strict cut-off of 0.50 gives almost perfect precision but misses most related questions (recall 0.298); the original permissive value of 1.15 finds most related questions (recall 0.917) but shows 665 wrong questions across the test set and answers all 15 queries that should match nothing. The F1 score peaks at 0.757 at a cut-off of 0.75, where a correct question is found for 44 of 46 queries and only 1 of 15 nothing-queries is answered. The cut-off was therefore changed from 1.15 to 0.75. Because the set is small and was prepared for this project, 0.75 is a sound choice for these papers but should be re-measured for another university.

C. Response time

TABLE V. Response time by question type (CPU only).

Question type	Runs	Median (s)	Range (s)
Out-of-syllabus (declined)	3	0	0
Exam facts	11	25	23–112
Explanation of a topic	12	77	51–214
Comparison of two topics	2	131	104–157
Lab program	2	193	173–212

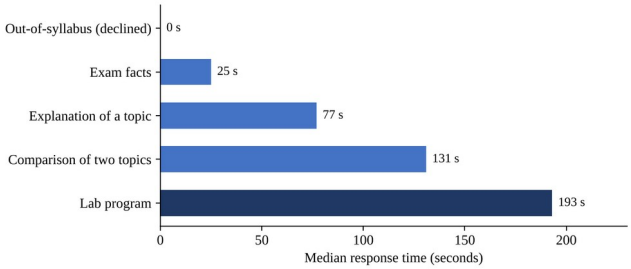


Fig. 10. Median response time by question type.

Declined questions are instant because the guard stops them before any model call, and exam-fact questions take about 25 s because the model only parses the question while code produces the counts. Explanations and lab programs take one to three and a half minutes because the model must generate long text on a CPU. A cached answer returns in seconds. These times are slower than the 3–6 s reported for a larger model on an Apple-silicon laptop [12], which reflects the Intel CPU and the long, structured answers; the persisted indexes and the answer cache play the same latency-saving role as the persisted index reported in [13].

D. Repeated questions and solutions bundle

The Most Repeated Questions view lists questions asked in two or more sittings of a subject and bundles a model answer for each into one PDF. Of the 125 repeated questions, 67 have curated answers (drafted with an AI assistant, checked, and numerical results recomputed in code) and 58 were generated offline by the local 7B model from the notes (Fig. 9). The local model was unreliable on calculation and proof questions: 41 of its first 99 answers were wrong or did not solve the question and were replaced by curated ones. The 58 that remain are mostly descriptive theory and were only skimmed.

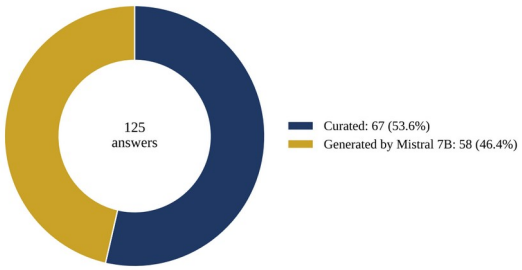


Fig. 9. Source of the 125 repeated-question answers.

E. Functional and regression testing

The 27 manual test cases are grouped in Table VI. Defects found during testing were corrected and the affected tests repeated; examples are a topic carried over from the previous message into an unrelated question, a trend chart that showed another subject's data, and a lab-program answer that was not enclosed in a code block.

TABLE VI. Manual test cases by area.

Area	Cases	Example check
Accounts and roles	7	Lockout after five wrong passwords
Study questions	11	Out-of-syllabus topic is declined
Exam Intelligence	4	Trend needs a selected subject
Question Paper Solver	1	Questions extracted and answered
OCR	1	Scanned page read with OCR
Repeatability	1	Same question, same answer
Rebuild and data safety	2	Corrupt file backed up

F. Worked examples

Exam history. For the question “what is queue? linear and circular, how many times it appeared” with the subject DS selected, uniRAG reported that the topic appeared in 5 sittings (6 questions) in 2025, 2024, 2023, 2022 and 2020, a coverage of 100 × 5/6 ≈ 83 %, listed the six questions with year and marks, and gave the priority tier “High exam priority (repeated topic)”. The numbers were identical on repeated runs.

Year-by-year trend. For the topic “Queue” the trend showed 2 matching questions in 2020, none in 2021 and one in each of 2022 to 2025, classified as stable. An early version showed the Data Structures chart when Computer Networks was selected, because the matcher fell back to the whole bank; the trend now uses only the selected subject.

Out-of-syllabus questions. “What is a Fenwick tree”, “explain Kubernetes” and “explain the French revolution” were each declined without calling the model, while “what is health” was answered because the word occurs in the cryptography notes.

G. Discussion

TABLE VII. uniRAG compared with a general chatbot.

Aspect	General chatbot	uniRAG
Internet	Required	Not needed
Privacy	Questions sent out	Data stays local
Answer source	General knowledge	University notes and papers
Out of syllabus	Answered	Declined
Exam statistics	Manual or invented	Counted by code
Scanned papers	Read by hand	OCR to question bank
Cost	Subscription or API fee	Free, open source

Separating the work of the model from the work of the code gave numbers that are exact, repeatable and traceable to the question bank, which a general chatbot cannot guarantee. The price is that counting is literal: a topic worded differently in a paper can be missed, a deliberate trade-off against invented counts. In the solutions bundle, the local model was unreliable on calculation and proof questions, since 41 of its first 99 answers had to be replaced, so descriptive explanations are more reliable than numerical ones. This agrees with the general finding that retrieval reduces but does not remove hallucination [4], and it is why uniRAG lists the source files with every answer and treats answers as study aids.

VIII. LIMITATIONS AND THREATS TO VALIDITY

The system was evaluated on one programme of one university, with a small labelled set prepared by the author and without a user study, so the results show feasibility, not general performance. Unlike [7] and [11], no automatic faithfulness metric was computed; faithfulness was checked through the guards and the regression suite. The 7B model can still state a wrong fact inside an explanation, OCR leaves occasional noise [18], [19], some questions have no detected year and are left out of year-wise counts, figures and diagrams are not extracted, and CPU inference is slow for long batches. The application is meant for single-machine use and is not hardened for internet deployment.

IX. CONCLUSION AND FUTURE WORK

This paper presented uniRAG, an offline retrieval-augmented study assistant for MCA students of Bangalore University. Its main finding is that the unreliable parts of a small local language model can be removed by design: the model is asked only to write explanations from retrieved notes, while counts, years, marks and trends are computed by code from a structured question bank of 1,426 questions, and guards decline questions that the notes or the syllabus do not support. This matches the evidence in the literature that retrieval

alone neither prevents hallucination [4] nor counts reliably across a corpus [16], [17].

The evaluation on real university material gave three main results. First, choosing the semantic matching cut-off by measurement on 61 labelled queries, instead of the permissive default of 1.15, raised the F1 score from 0.435 to 0.757 and cut wrongly shown questions from 665 to 61. Second, the system answers exam-fact questions in about 25 s and explanations in one to three and a half minutes on a laptop with no GPU, and returns saved answers in seconds. Third, all numbers shown to a student were repeatable and traceable to the question bank.

The work has limits. It covers one programme of one university, the labelled set is small, no student user study was done, the 7B model can still slip on a fact inside an explanation, and OCR noise and literal counting can hide some questions. Future work should add sentence-level verification of explanations against the notes, evaluation with faithfulness metrics such as Ragas [7], faster inference through a GPU or a quantised model, extraction of figures and tables, Kannada and Hindi support, a secure campus deployment with e-mail verification, and a larger multi-annotator evaluation with a study among students.

REFERENCES

[1] Ministry of Education, Government of India, *All India Survey on Higher Education 2021-22*. [Online]. Available: https://education.gov.in/sites/upload_files/mhrd/files/statistics-new/AISHE%20Book_2021-22_4.pdf

[2] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," arXiv:2005.11401, 2020.

[3] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2023.

[4] L. Huang *et al.*, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Trans. Inf. Syst.*, arXiv:2311.05232, 2025.

[5] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.

[6] A. Asai, Z. Wu, Y. Wang, A. Sil and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. ICLR*, 2024, arXiv:2310.11511.

[7] S. Es, J. James, L. Espinosa-Anke and S. Schockaert, "Ragas: Automated evaluation of retrieval augmented generation," arXiv:2309.15217, 2023.

[8] X. Li, Y. Cao, Y. Ma and A. Sun, "Long context vs. RAG for LLMs: An evaluation and revisits," arXiv:2501.01880, 2024.

[9] G. Lang and T. Gurpinar, "AI-powered learning support: A study of retrieval-augmented generation (RAG) chatbot effectiveness in an online course," *Inf. Syst. Educ. J.*, vol. 23, no. 2, pp. 4–13, Mar. 2025.

[10] S. Kahl *et al.*, "Evaluating the impact of advanced LLM techniques on AI-lecture tutors for a robotics course," arXiv:2408.04645, 2024.

[11] A. Ostrowska *et al.*, "From surface learning to deep understanding: A grounded AI tutoring system for Moodle," arXiv:2605.06963, 2026.

[12] T. Takamura and A. Umezawa, "Privacy-preserving retrieval-augmented generation on local devices for regenerative medicine applications," medRxiv, doi: 10.1101/2025.10.20.25337146, 2025.

[13] A. S. Lee, "Offline RAG with small language models optimized for resource-constrained environments: Natural disaster safety advice," poster, NeurIPS 2026 Affinity Event, 2026.

[14] P. Devine, "ALoFTRAG: Automatic local fine tuning for retrieval augmented generation," arXiv:2501.11929, 2025.

[15] R. Dayarathne, U. Ranaweera and U. Ganegoda, "Comparing the performance of LLMs in RAG-based question-answering: A case study in computer science literature," arXiv:2511.03261, 2025.

[16] Q. Luo, X. Li, T. Fan, X. Chen and X. Qiu, "Towards global retrieval augmented generation: A benchmark for corpus-level reasoning," arXiv:2510.26205, 2025.

[17] H. Zhu *et al.*, "Aggregation queries over unstructured text: Benchmark and agentic method," arXiv:2602.01355, 2026.

[18] J. Zhang *et al.*, "OCR hinders RAG: Evaluating the cascading impact of OCR on retrieval-augmented generation," arXiv:2412.02592, 2025.

[19] L. Sun *et al.*, "When good OCR is not enough: Benchmarking OCR robustness for retrieval-augmented generation," arXiv:2605.00911, 2026.

[20] P. Prasanjith, P. B. More, A. Kunchukuttan and R. Dabre, "IndicRAGSuite: Large-scale datasets and a benchmark for Indian language RAG systems," arXiv:2506.01615, 2025.

[21] J. Johnson, M. Douze and H. Jégou, "Billion-scale similarity search with GPUs," arXiv:1702.08734, 2017.

[22] Z. Nussbaum *et al.*, "Nomic Embed: Training a reproducible long context text embedder," arXiv:2402.01613, 2024.

[23] A. Q. Jiang *et al.*, "Mistral 7B," arXiv:2310.06825, 2023.

[24] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.

[25] K. Moriarty, B. Kaliski and A. Rusch, "PKCS #5: Password-based cryptography specification version 2.1," RFC 8018, IETF, 2017.